# Technical Design Report: UniEvent

**Course:** CE 308/408 Cloud Computing

**Student Name:** M. Mussab Altaf

Ghulam Ishaq Khan Institute of Engineering Sciences and Technology

**Date:** March 28, 2026

---

**1. Executive Summary**

The UniEvent project is a cloud-native web application designed to serve as a centralized hub for university events. To ensure the system meets the high demands of student registration drives and festivals, the architecture is built on Amazon Web Services (AWS) focusing on four pillars: **Security, Scalability, Fault Tolerance, and High Availability.** This report details the implementation of a tiered network architecture using VPC, EC2, ELB, and S3.

**2. External Data Integration**

**2.1 Selected API: Ticketmaster Discovery API**

For the automated ingestion of event data, the **Ticketmaster Discovery API** was selected.

- **Justification:** It provides structured JSON data including event titles, local dates, venue details, and high-resolution image URLs.
- **Data Flow:** The application on EC2 instances performs periodic GET requests to the API. The retrieved JSON is parsed, and media (posters) are cached in S3 to ensure the system remains functional even if the external API faces rate-limiting or downtime.

---

**3. Architectural Components and Justification**

**3.1 Network Topology (VPC & Subnets)**

The foundation of UniEvent is a custom **Virtual Private Cloud (VPC)** designed with a Multi-AZ (Availability Zone) strategy.

- **Public Subnets:** Host the Application Load Balancer (ALB) and NAT Gateways. These are the only components with direct routes to the Internet Gateway (IGW).
- **Private Subnets:** Host the application logic (EC2 instances). This ensures that the servers containing university data are not reachable via a public IP address, mitigating a significant portion of external cyber threats.
- **Connectivity:** To allow private instances to fetch event data from the Ticketmaster API, a **NAT Gateway** is deployed in the Public Subnet, providing a secure outbound-only connection.

### 3.2 Compute and High Availability (EC2 & ELB)

To fulfill Requirement #6 (fault tolerance), the compute layer is decoupled from the entry point.

- **Elastic Load Balancing (ELB):** An Application Load Balancer is deployed to receive all incoming student traffic (HTTP/HTTPS). It performs health checks on the EC2 instances.
- **Multi-AZ Deployment:** EC2 instances are distributed across at least two Availability Zones. If a hardware failure occurs in one zone, the ALB automatically shifts traffic to the instances in the healthy zone, ensuring zero downtime for the UniEvent platform.

### 3.3 Storage Layer (Amazon S3)

**Amazon S3** is utilized as the primary storage for both application metadata and media.

- **Persistence:** Rather than storing event data in local instance memory (which is lost if an instance is terminated), the data is stored in S3 buckets.
- **Security:** Public access to the S3 bucket is blocked. Media is served via signed URLs or through the web application layer to ensure posters and student-uploaded images are protected.

---

## 4. Security Framework

Security is integrated into every layer of the UniEvent architecture.

### 4.1 IAM (Identity and Access Management)

Instead of using long-term access keys (which are a security risk), the application uses **IAM Roles**.

- An **EC2 Instance Profile** is created with a policy granting s3:PutObject and s3:GetObject permissions.

- This role is attached to the EC2 instances, allowing them to communicate with S3 securely without any credentials stored in the source code.

### 4.2 Security Groups (Stateful Firewalls)

A "Chain of Trust" is established using Security Groups: | Tier | Source | Protocol | Port | Purpose | **ALB SG** | 0.0.0.0/0 | TCP | 80/443 | Allow student web traffic. | | **EC2 SG** | **ALB Security Group** | TCP | 80 | **Only** allow traffic from the Load Balancer. | | **S3 Policy** | VPC Endpoint / IAM | HTTPS | 443 | Secure data storage access. |

---

## 5. System Workflow (Step-by-Step)

1. **Request:** A student accesses the UniEvent URL; the request hits the **ALB**.
2. **Routing:** The ALB routes the request to a healthy **EC2 instance** in a private subnet.
3. **Ingestion:** The EC2 application periodically calls the **Ticketmaster API** via the **NAT Gateway**.
4. **Storage:** Fetched event posters are uploaded to **S3** for fast, reliable delivery.
5. **Response:** The application renders the "University Events" page, pulling data from S3 and displaying it to the student.

## 6. Conclusion

The proposed architecture for UniEvent adheres to the AWS Well-Architected Framework. By utilizing a private-subnet strategy and Multi-AZ load balancing, the system achieves the university's goal of a secure, fault-tolerant platform capable of handling peak registration loads.